THE UNIVERSITY OF AZAD JAMMU AND KASHMIR, MUZAFFARABAD

Software Testing

LAB TASK 1



Submitted to: Engr.Fahad Nisar

Submitted by: Faizan Shabbir Abbasi

2024-SET-21

Table of Contents

1. Objective 3
2. Introduction 3
3. Requirements 3
4. Practical Work 3
    Step 1: Install Node.js 3
    Step 2: Verify Node.js and npm 4
    Step 3: Create the Project Folder 5
    Step 4: Initialize the Node.js Project 5
    Step 5: Install and Initialize Playwright 6
    Step 6: Inspect the Project Files 7
    Step 7: Run the Sample Tests 8
    Step 8: View the HTML Report 9
5. Troubleshooting 9
6. Result 9
7. Conclusion 10
8. References 10

1. Objective

The objective of this lab task is to install Node.js and npm, create a Node.js project, set up Playwright using JavaScript, run the sample tests, and view the HTML test report.

2. Introduction

Node.js is a runtime that allows JavaScript to run outside a web browser. npm is the package manager used to install and manage project packages. Playwright is a testing tool used to automate browsers and test web applications. In this practical, these tools are installed and tested on Windows 11 using Command Prompt.

3. Requirements

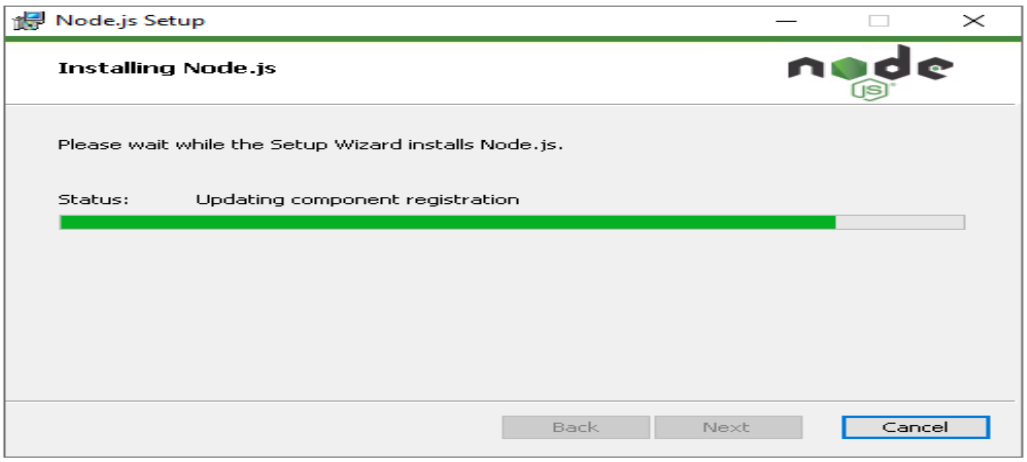
- Windows 11 computer
- Internet connection
- Permission to install software
- Node.js LTS version
- Command Prompt
- Playwright and its supported browser

4. Practical Work

Step 1: Install Node.js

I opened the official Node.js download page at https://nodejs.org/en/download and downloaded the Windows Installer (.msi) for the LTS release. I ran the installer, accepted the license, and kept the default options, including npm and Add to PATH. I completed the installation.

Screenshot: Node.js installation



Alt text: Node.js LTS installer or completed installation on Windows.

What this screenshot shows: This screenshot provides evidence that Node.js was installed.

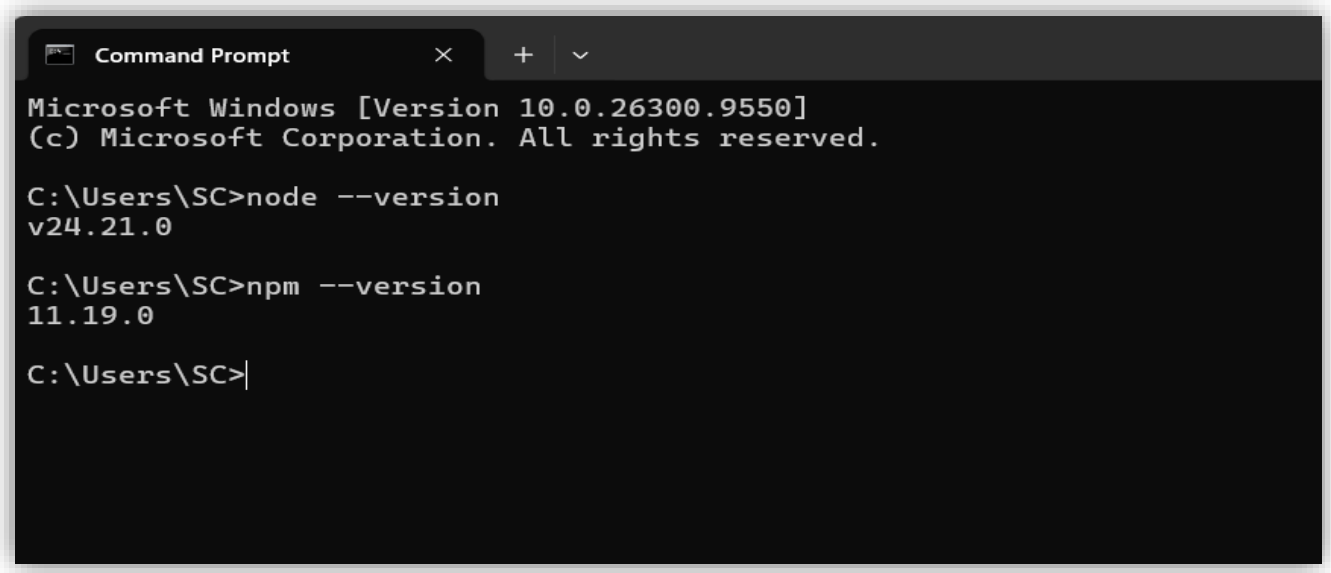
Step 2: Verify Node.js and npm

I opened a new Command Prompt window and entered these commands one at a time:

```
node --version
npm --version
```

Both commands should display version numbers. This confirms that Node.js and npm are available from Command Prompt.

Screenshot: Node.js and npm versions



Alt text: Command Prompt showing the node --version and npm --version commands with their version numbers.

What this screenshot shows: The version numbers confirm that Node.js and npm are installed.

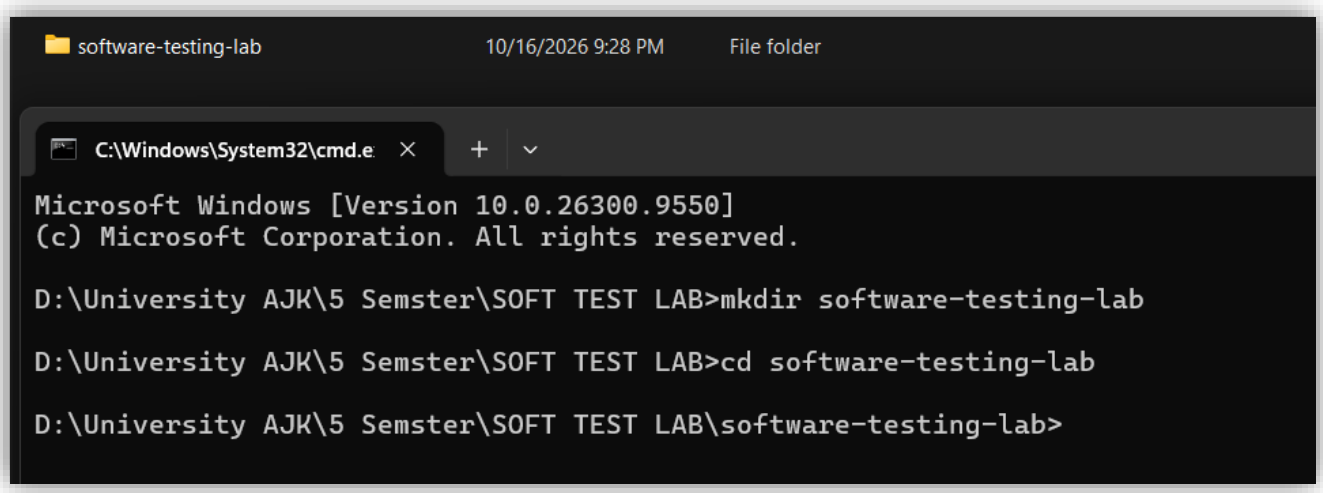
Step 3: Create the Project Folder

I created a folder for the lab project and moved into it by running:

```
mkdir software-testing-lab
cd software-testing-lab
```

The project folder keeps the files for this practical in one place.

Screenshot 3: Project folder creation



Alt text: Command Prompt showing the software-testing-lab folder being created and opened.

What this screenshot shows: The commands create the folder and set it as the current working directory.

Step 4: Initialize the Node.js Project

Inside the project folder, I ran the following command:

```
npm init -y
```

This command creates a package.json file with default project information. The file is used to describe the project and manage its packages.

Screenshot 4: Node.js project initialization

```
D:\University AJK\5 Semster\SOFT TEST LAB\software-testing-lab>npm init -y
Wrote to D:\University AJK\5 Semster\SOFT TEST LAB\software-testing-lab\package.json:

{
  "name": "software-testing-lab",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

Alt text: Command Prompt showing npm init -y completed successfully inside the software-testing-lab folder.

What this screenshot shows: The command initializes the Node.js project and creates package.json.

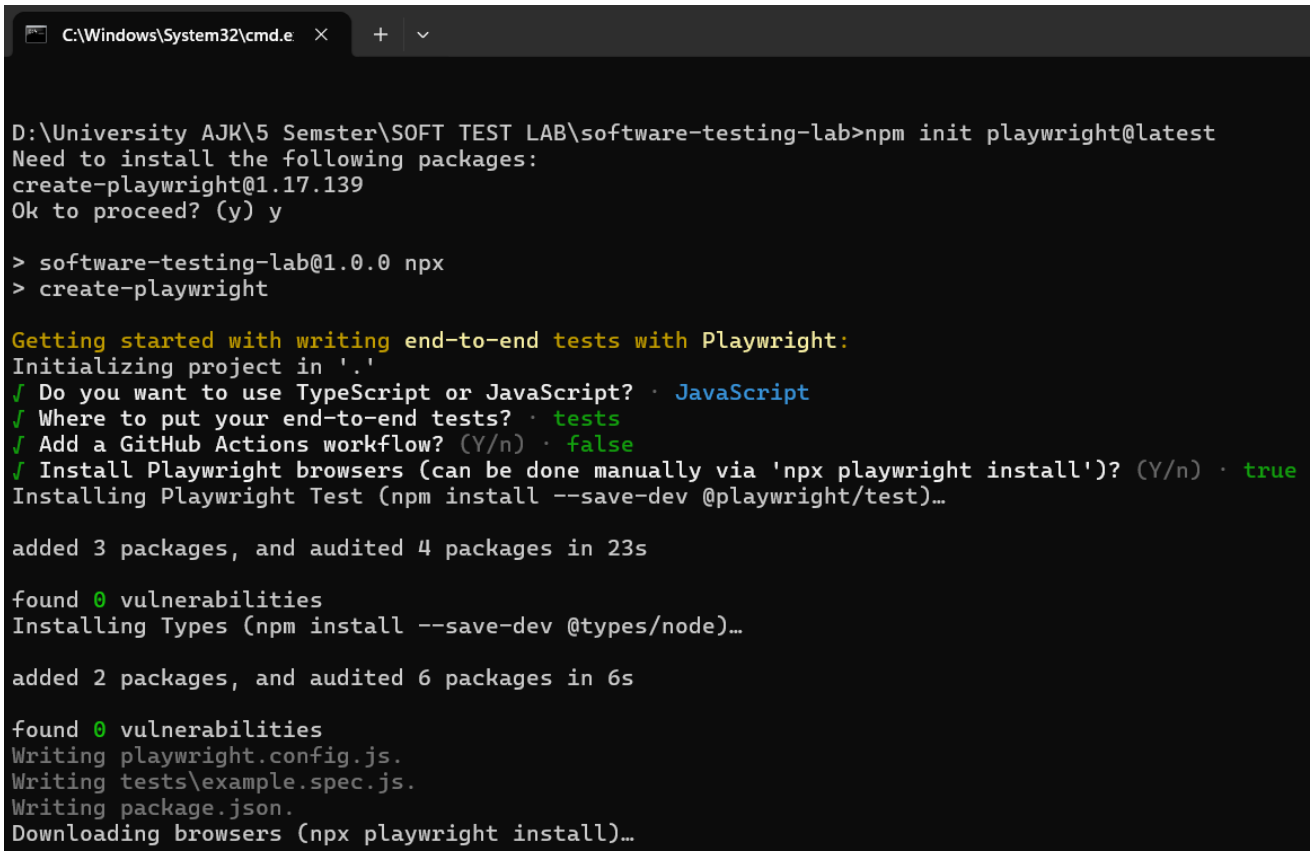
Step 5: Install and Initialize Playwright

I ran the Playwright initializer:

```
npm init playwright@latest
```

If prompted, I entered y to install the initializer. I selected JavaScript, used tests as the test folder, selected No for GitHub Actions, and selected Yes to install Playwright browsers. I waited for the installation to finish.

Screenshot 5: Playwright setup



Alt text: Command Prompt showing the Playwright setup choices and successful installation.

What this screenshot shows: This shows that Playwright was configured for a JavaScript project.

Step 6: Inspect the Project Files

After installation, I checked the project files. The important files and folders are:

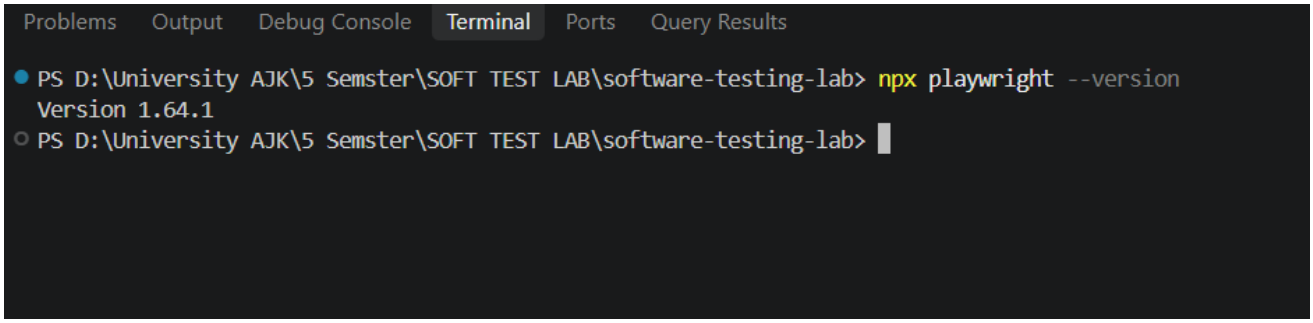
- package.json — contains project information and package scripts/dependencies.
- package-lock.json — records the resolved package versions.
- node_modules — contains installed packages.
- playwright.config.js — contains Playwright test settings.
- tests/example.spec.js — contains the generated sample tests.

I then checked the installed Playwright version with this command:

```
npx playwright --version
```

The command should display the installed Playwright version.

Screenshot 6: Project files and Playwright version



Alt text: Project folder showing package.json, package-lock.json, node_modules, playwright.config.js, and tests, with the Playwright version command visible if possible.

What this screenshot shows: This confirms that the expected project files exist and Playwright is installed.

Step 7: Run the Sample Tests

I ran the generated sample tests using:

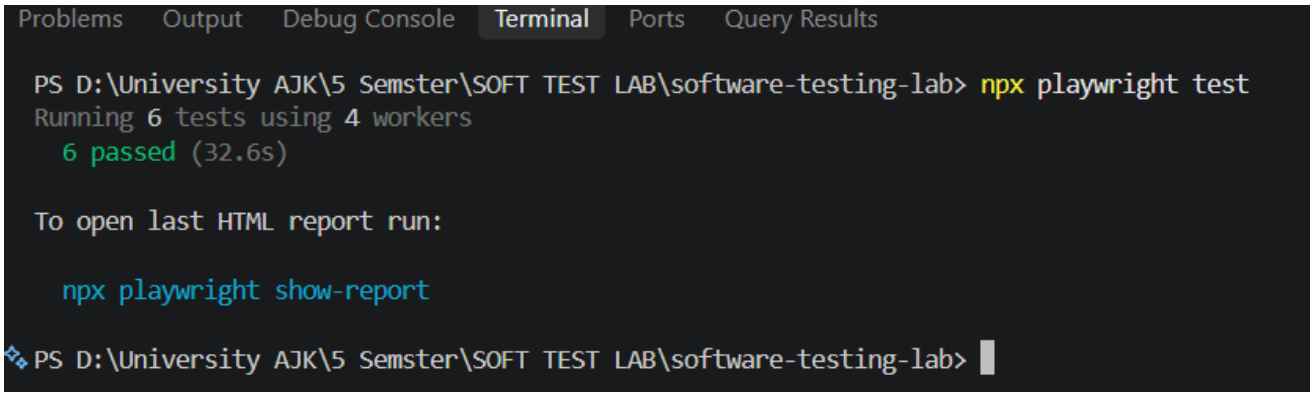
```
npx playwright test
```

The command runs the tests in headless mode by default, meaning the browser window is not normally shown. The terminal should display the test results. To watch the tests run in Chromium, I can use:

```
npx playwright test --project=chromium --headed
```

A successful run should show that the sample tests passed. The exact number of tests may depend on the generated example and installed version.

Screenshot 7: Successful Playwright test run



Alt text: Command Prompt showing npx playwright test and the final summary of passing tests.

What this screenshot shows: The final summary shows whether the generated sample tests passed.

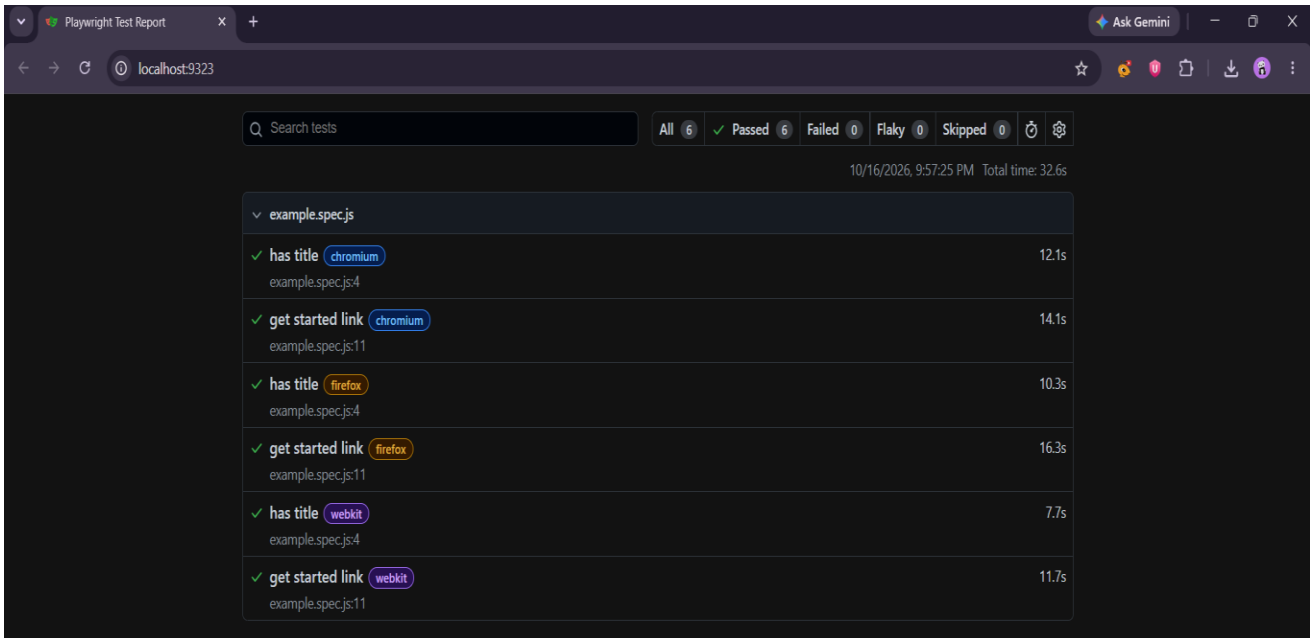
Step 8: View the HTML Report

After the tests finish, I opened the HTML report by running:

```
npx playwright show-report
```

The report opens in a web browser and provides a visual summary of the test results. I reviewed the report and pressed Ctrl + C in Command Prompt to stop the report server when finished.

Screenshot 8: Playwright HTML report



Alt text: Browser window showing the Playwright HTML report with the sample test results.

What this screenshot shows: The report provides a readable summary of the test run.

5. Troubleshooting

- If node or npm is not recognized, open a new Command Prompt window and check that Node.js was installed with Add to PATH enabled.
- If browser executables are missing, run npx playwright install.
- Keep the internet connected because the sample tests may visit an online website.
- If a test fails, read the error message and check the browser installation and internet connection.

6. Result

Node.js and npm were installed and verified. A Node.js project was initialized, Playwright was configured with JavaScript, the sample tests were run, and the HTML report was opened. The screenshots in this report provide evidence of the steps completed.

7. Conclusion

This lab task introduced the basic setup of a JavaScript testing environment. I learned how to install Node.js, use npm to initialize a project, configure Playwright, execute browser tests, and view test results in an HTML report.

8. References

Node.js download page: https://nodejs.org/en/download

Playwright documentation: https://playwright.dev/docs/intro

Thank you so much